

버튼·LCD 통합 실습

VS Code 사전 시뮬레이션 → 오픈소스 CLI → 보드 실험

같은 RTL에 10개 회로를 넣고 버튼으로 선택한다. LCD에 모드 번호와 회로 이름을 표시한다.

작성일 2026. 09. 10. · 이해리

1. 먼저 2,560개 입력과 버튼·LCD 동작을 검사한다.
2. 합성·배치배선·비트스트림을 생성한다.
3. 보드에서 모드를 순환하고 사진·영상으로 설명한다.

통합 실습 목차

1부 · VS Code 사전 시뮬레이션
clone·새 창·workspace·실행·파일형

2부 · 통합 회로 이해
10개 모드·버튼·LCD·보드 연결

3부 · 구현과 결과
CLI 설치·실행·검증 범위
보드·사진·영상·실험 후 레포트

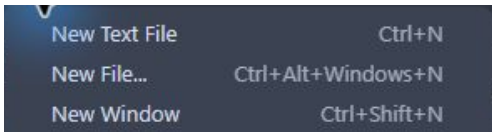
전체 목차 PDF

템플릿과 새 창

Git·Python 3·VS Code와 Icarus Verilog·Yosys·openXC7을 준비한다.

```
git clone https://github.com/Glaysia/fpga-lab-template.git
```

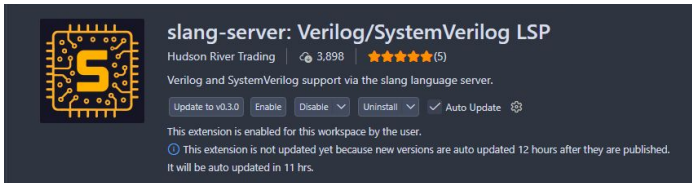
VS Code: File → New Window → Open Workspace from File...



```
opensource_cli/integrated/opensource_cli_integrated.code-workspace
```

확장과 Explorer

Extensions에서 slang-server와 VaporView를 Install 또는 Enable (Workspace).



Explorer에서 COMMON → rtl → lab1_integrated.v를 두 번 누른다.

COMMON → tb → tb_lab1_integrated.sv, constraints → lab1_integrated.xdc도 연다.

PROJECT에는 project.json·workspace·결과 폴더가 있다. File → Save All.

통합 RTL · 모드 번호

lab1_integrated.v · 1-11행 · 실제 VS Code 화면

```
1 // Input clock MUST be the trainer's 1 kHz main clock (B6).
2 // Mode button N8; reset K4; data DIPSW1..8 = sw[7]..sw[0].
3 module lab1_integrated #(parameter integer DEBOUNCE_CYCLES=20, LCD_POWER_WAIT=50)(
4     input wire clk,rst,mode_button, input wire [7:0] sw,
5     output reg [7:0] led, output wire [7:0] seg_data,
6     output wire lcd_e,lcd_rs,lcd_rw, output wire [7:0] lcd_data);
7     wire press; reg [3:0] mode;
8     button_onepulse #(DEBOUNCE_CYCLES) button(clk,rst,mode_button,press);
9     always @(posedge clk or posedge rst)
10         if(rst) mode<=0;
11         else if(press) mode<=(mode==9)?0:mode+1;
```

기본값은 디바운스 20클록·LCD 대기 50클록이다. 입력 클록은 보드의 1kHz다.

reset은 mode=0, 버튼 펄스는 다음 모드로 이동한다. 내부 번호 0-9는 LCD의 01-10에 대응한다.

통합 RTL · 논리 · 가산 연결

lab1_integrated.v · 12-18행 · 실제 VS Code 화면

```
12 wire gx,gy,gz,fs,fc,ac,borrow,mz;
13 wire [3:0] sum,difference;
14 wire [2:0] comparison,encoded;
15 wire [7:0] demuxed,decoded,hex_segments;
16 logic_gate gates(sw[7],sw[6],gx,gy,gz);
17 full_adder fa(sw[7],sw[6],sw[5],fs,fc);
18 adder_4bit add4(sw[7:4],sw[3:0],sum,ac);
```

공유 RTL의 논리 게이트·전가산기·4비트 가산기를 인스턴스화한다. DIP 입력에서 필요한 비트를 연결한다.

통합 RTL · 나머지 회로 연결

lab1_integrated.v · 19-25행 · 실제 VS Code 화면

```
19 sub_4bit sub4(sw[7:4],sw[3:0],difference,borrow);
20 compare_4 comp(sw[7:4],sw[3:0],comparison);
21 mux_4x1 mux(sw[7:4],sw[1:0],mz);
22 demux_1x8 demux(sw[7],sw[2:0],demuxed);
23 encoder8x3 encoder(sw,encoded);
24 decoder3x8 decoder(sw[2],sw[1],sw[0],decoded);
25 seg_decoder seven(sw[3:0],hex_segments);
```

감산·비교·MUX·DEMUX·인코더·디코더·7세그먼트도 같은 입력 스위치를 사용한다.

모든 회로는 함께 동작하며, 다음 코드에서 현재 모드의 출력을 선택한다.

통합 RTL · 출력 선택 01-05

lab1_integrated.v · 26-34행 · 실제 VS Code 화면

```
26 assign seg_data=(mode==9)?hex_segments:8'b0;  
27 always @* begin  
28     led=0;  
29     case(mode)  
30         0:led={gx,gy,gz,5'b0};  
31         1:led={fc,fs,6'b0};  
32         2:led={ac,sum,3'b0};  
33         3:led={borrow,difference,3'b0};  
34         4:led={comparison,5'b0};
```

7세그먼트는 mode=9에서만 켜다. LED 기본값을 0으로 두고 모드별 결과를 지정한다.

비트 묶음의 왼쪽이 led[7]이며 보드 LED1에 연결된다. 남은 비트는 0으로 채운다.

통합 RTL · 출력 선택 06-10

lab1_integrated.v · 35-44행 · 실제 VS Code 화면

```
35     5:led={mz,7'b0};
36     6:led=demuxed;
37     7:led={encoded,5'b0};
38     8:led=decoded;
39     9:led=hex_segments;
40     default:led=0;
41   endcase
42   end
43   lcd_modes #(LCD_POWER_WAIT) display(clk,rst,mode,lcd_e,lcd_rs,lcd_rw,lcd_data);
44 endmodule
```

MUX부터 7세그먼트까지 출력 선택을 마친다. LCD에는 같은 mode를 전달해 회로 이름을 표시한다.

버튼 RTL · 두 단계 동기화

button_onepulse.v · 1-9행 · 실제 VS Code 화면

```
1 module button_onepulse #(parameter integer STABLE_CYCLES=20)(
2     input wire clk,rst,button, output reg pulse);
3     (* ASYNC_REG="TRUE" *) reg sync1, sync2;
4     reg accepted;
5     integer count;
6     always @(posedge clk or posedge rst) begin
7         if(rst) begin sync1<=0; sync2<=0; end
8         else begin sync1<=button; sync2<=sync1; end
9     end
```

외부 버튼을 sync1, sync2 두 플립플롭을 거쳐 받는다. ASYNC_REG 속성으로 동기화 레지스터임을 표시한다.

버튼 RTL · 한 번만 전환

button_onepulse.v · 10-20행 · 실제 VS Code 화면

```
10  always @(posedge clk or posedge rst) begin
11      if(rst) begin count<=0; accepted<=0; pulse<=0; end
12      else begin
13          pulse<=0;
14          if(sync2==accepted) count<=0;
15          else if(count==STABLE_CYCLES-1) begin
16              accepted<=sync2; count<=0; pulse<=sync2;
17          end else count<=count+1;
18      end
19  end
20  endmodule
```

입력이 accepted와 다르게 20클럭 유지되면 새 상태를 받아들인다. 그 전에는 연속 안정 시간을 센다.

pulse의 기본값은 0이고 누름을 받아들일 때만 1이다. 땔 때는 상태만 바뀌어 모드를 넘기지 않는다.

LCD RTL · 포트와 상태

lcd_modes.v · 1-11행 · 실제 VS Code 화면

```
1  // Board source: 2022 CHARACTER LCD guide p13/p30, 1 kHz clock, 16x2, 8-bit bus.  
2  // Each byte has 1 ms setup, 1 ms enable-high and >=2 ms recovery.  
3  module lcd_modes #(parameter integer POWER_WAIT=50)(  
4      input wire clk,rst, input wire [3:0] mode,  
5      output reg lcd_e,lcd_rs, output wire lcd_rw, output reg [7:0] lcd_data);  
6      reg [1:0] phase;  
7      integer power_count;  
8      reg [5:0] index;  
9      reg [3:0] shown_mode;  
10     reg [127:0] name;  
11     assign lcd_rw=1'b0;
```

phase는 한 바이트의 전송 단계, index는 화면 내 위치다. shown_mode는 현재 화면의 모드다.

한 화면을 전송하는 동안 shown_mode를 유지하여 두 줄의 모드가 섞이지 않게 한다.

LCD RTL · 회로명 선택

lcd_modes.v · 12-21행 · 실제 VS Code 화면

```
12  always @* begin
13      case(shown_mode)
14          0:name="AND OR XOR      "; 1:name="FULL ADDER      ";
15          2:name="4 BIT ADDER     "; 3:name="4 BIT SUBTRACT  ";
16          4:name="4 BIT COMPARE   "; 5:name="4 TO 1 MUX      ";
17          6:name="1 TO 8 DEMUX     "; 7:name="8 TO 3 ENCODER   ";
18          8:name="3 TO 8 DECODER  "; 9:name="HEX 7 SEGMENT  ";
19          default:name="RESET REQUIRED ";
20      endcase
21  end
```

shown_mode에 따라 16문자 회로명을 고른다. 남은 자리는 공백으로 채운다.

한 화면을 전송하는 동안 shown_mode를 유지하여 두 줄의 모드가 섞이지 않게 한다.

LCD RTL · 리셋과 전원 대기

lcd_modes.v · 22-26행 · 실제 VS Code 화면

```
22  always @(posedge clk or posedge rst) begin
23      if(rst) begin
24          phase<=0; power_count<=0; index<=0; shown_mode<=0;
25          lcd_e<=0; lcd_rs<=0; lcd_data<=0;
26      end else if(power_count<POWER_WAIT) power_count<=power_count+1;
```

리셋 때 모든 상태와 출력 신호를 초기화한다. 1kHz에서 POWER_WAIT=50은 50ms 대기다.

한 화면을 전송하는 동안 shown_mode를 유지하여 두 줄의 모드가 섞이지 않게 한다.

LCD RTL · 초기 명령

lcd_modes.v · 27-34행 · 실제 VS Code 화면

```
27  else case(phase)
28      0:begin
29          lcd_e<=0; lcd_rs<=0;
30          case(index)
31              0,1,2:lcd_data<=8'h38;
32              3:lcd_data<=8'h0c;
33              4:lcd_data<=8'h06;
34              5:begin lcd_data<=8'h80; shown_mode<=mode; end
```

38을 세 번 보낸 뒤 0C, 06으로 초기화한다. 첫 줄 주소 80을 보낼 때 mode를 shown_mode에 저장한다.

한 화면을 전송하는 동안 shown_mode를 유지하여 두 줄의 모드가 섞이지 않게 한다.

LCD RTL · MODE 01-10

lcd_modes.v · 35-42행 · 실제 VS Code 화면

```
35: 6:begin lcd_rs<=1; lcd_data<="M"; end
36: 7:begin lcd_rs<=1; lcd_data<="O"; end
37: 8:begin lcd_rs<=1; lcd_data<="D"; end
38: 9:begin lcd_rs<=1; lcd_data<="E"; end
39: 10:begin lcd_rs<=1; lcd_data<=" "; end
40: 11:begin lcd_rs<=1; lcd_data<=(shown_mode==9)?"1":"0"; end
41: 12:begin lcd_rs<=1; lcd_data<=(shown_mode==9)?"0":("1"+shown_mode); end
42: 22:lcd_data<=8'hc0;
```

문자 MODE와 두 자리 번호를 보낸다. 내부 mode=9만 10으로 표시하고, 그다음 C0으로 둘째 줄을 선택한다.

한 화면을 전송하는 동안 shown_mode를 유지하여 두 줄의 모드가 섞이지 않게 한다.

LCD RTL · 이름의 한 글자

lcd_modes.v · 43-49행 · 실제 VS Code 화면

```
43  default:begin
44      lcd_rs<=1;
45      if(index>=23 && index<=38) lcd_data<=name[127-(index-23)*8 -: 8];
46      else lcd_data<=" ";
47      end
48  endcase
49  phase<=1;
```

index=23-38에서 128비트 이름을 8비트씩 꺼낸다. 나머지 첫 줄 위치는 공백으로 채운다.

한 화면을 전송하는 동안 shown_mode를 유지하여 두 줄의 모드가 섞이지 않게 한다.

LCD RTL · E 펄스와 반복

lcd_modes.v · 50-60행 · 실제 VS Code 화면

```
50     end
51     1:begin lcd_e<=1; phase<=2; end
52     2:begin lcd_e<=0; phase<=3; end
53     3:begin
54         // First function-set recovery is 6 ms including setup of the next byte.
55         if(index==0 && power_count<POWER_WAIT+2) power_count<=power_count+1;
56         else begin phase<=0; index<=(index==38)?5:index+1; end
57     end
58 endcase
59 end
60 endmodule
```

phase 1에서 E를 올리고 phase 2에서 내린다. 1kHz이므로 E high는 1ms다.

첫 초기 명령 뒤에는 추가 대기한다. 한 화면이 끝나면 index=5로 돌아가 모드를 새로 저장한다.

통합 TB · 검사 환경

tb_lab1_integrated.sv · 1-13행 · 실제 VS Code 화면

```
1 | timescale 1ns/1ps
2 | module tb_lab1_integrated;
3 |   reg clk=0,rst=1,mode_button=0; reg [7:0] sw=0;
4 |   wire [7:0] led,seg_data,lcd_data; wire lcd_e,lcd_rs,lcd_rw;
5 |   lab1_integrated #(.DEBOUNCE_CYCLES(4),.LCD_POWER_WAIT(5)) dut(.*);
6 |   always #5 clk=~clk;
7 |   integer checked=0,m,n,k,v,frames=0,addr=0;
8 |   reg [7:0] expected,expected_seg;
9 |   reg [127:0] line1,line2,expected_name,completed_line1,completed_line2;
10 |  reg [55:0] expected_heading;
11 |  integer init_count=0;
12 |  reg [7:0] init_words[0:4];
13 |  reg [7:0] digits[0:15];
```

10ns 클럭과 짧은 디바운스·전원 대기로 기능 검사를 빠르게 한다. 실제 합성의 1kHz 설정과 구분한다.

실험 전 레포트에는 이 코드의 역할과 자신의 실행 로그·파형을 함께 설명한다.

통합 TB · 반복 동작

tb_lab1_integrated.sv · 14~22행 · 실제 VS Code 화면

```
14 task cycles(input integer amount);  
15     repeat(amount) @(negedge clk);  
16 endtask  
17 task assert_mode(input integer value);  
18     if(dut.mode!=value) $fatal(1,"Mode expected %0d actual %0d",value,dut.mode);  
19 endtask  
20 task press_once;  
21     mode_button=1; cycles(12); mode_button=0; cycles(12);  
22 endtask
```

cycles는 클록 하강까지 기다리고 assert_mode는 모드 번호를 검사한다.
press_once는 눌렀다 떼는 한 번의 동작이다.

실험 전 레포트에는 이 코드의 역할과 자신의 실행 로그·파형을 함께 설명한다.

통합 TB · LCD 명령 검사

tb_lab1_integrated.sv · 23-32행 · 실제 VS Code 화면

```
23 // Decode the actual external LCD bus at E falling edge.
24 always @(negedge lcd_e) if(!rst) begin
25     if(lcd_rw!=0) $fatal(1,"LCD must write only");
26     if(!lcd_rs) begin
27         if(init_count<5) begin
28             if(lcd_data!=init_words[init_count]) $fatal(1,"LCD initialization command %0d got %h",init_count,
29                 lcd_data);
30             init_count=init_count+1;
31         end
32         if(lcd_data==8'h80) begin addr=0;line1=0;end
33         else if(lcd_data==8'hc0) begin addr=64;line2=0;end
```

실제 출력 lcd_e의 하강에서 lcd_rw와 초기 명령을 검사한다. 주소 80·C0을 만나면 해당 줄 조립을 시작한다.

실험 전 레포트에는 이 코드의 역할과 자신의 실행 로그·파형을 함께 설명한다.

통합 TB · 완성된 화면

tb_lab1_integrated.sv · 33-41행 · 실제 VS Code 화면

```
33   end else begin
34       if(addr<16) line1[127-addr*8 -: 8]=lcd_data;
35       if(addr>=64 && addr<80) line2[127-(addr-64)*8 -: 8]=lcd_data;
36       if(addr==79) begin
37           completed_line1=line1;completed_line2=line2;frames=frames+1;
38       end
39       addr=addr+1;
40   end
41 end
```

주소에 따라 받은 문자를 16문자 버퍼에 넣는다. 둘째 줄 마지막 문자까지 받은 순간 완성된 두 줄을 보관한다.

실험 전 레포트에는 이 코드의 역할과 자신의 실행 로그·파형을 함께 설명한다.

통합 TB · 리셋과 바운스

tb_lab1_integrated.sv · 42-52행 · 실제 VS Code 화면

```
42  initial begin
43      init_words[0]=8'h38;init_words[1]=8'h38;init_words[2]=8'h38;init_words[3]=8'h0c;init_words[4]=8'h06;
44      digits[0]=252;digits[1]=96;digits[2]=218;digits[3]=242;
45      digits[4]=102;digits[5]=182;digits[6]=190;digits[7]=224;
46      digits[8]=254;digits[9]=246;digits[10]=238;digits[11]=62;
47      digits[12]=156;digits[13]=122;digits[14]=158;digits[15]=142;
48      $dumpfile("wave.vcd");$dumpvars(0,tb_lab1_integrated);
49      cycles(3);rst=0;cycles(12);assert_mode(0);
50      // Bounces shorter than four samples must not be accepted.
51      repeat(3) begin mode_button=1;cycles(1);mode_button=0;cycles(2);end
52      cycles(12);assert_mode(0);
```

기대 LCD 명령·숫자 패턴을 준비한다. 1클록 누름과 2클록 땜을 반복해도 mode=0인지 확인한다.

실험 전 레포트에는 이 코드의 역할과 자신의 실행 로그·파형을 함께 설명한다.

통합 TB · 모드별 입력

tb_lab1_integrated.sv · 53-60행 · 실제 VS Code 화면

```
53  for(m=0;m<10;m=m+1) begin
54      assert_mode(m);
55  for(n=0;n<256;n=n+1) begin
56      sw=n;cycles(1);expected=0;expected_seg=0;
57      case(m)
58      0:expected={sw[7]&sw[6],sw[7]|sw[6],sw[7]^sw[6],5'b0};
59      1:expected=(int'(sw[7])+int'(sw[6])+int'(sw[5]))<<6;
60      2:expected=((n/16)+(n%16))<<3;
```

10개 모드 각각에서 DIP 256개를 모두 넣는다. 출력의 비트 배치까지 포함하여 별도 기대값을 계산한다.

실험 전 레포트에는 이 코드의 역할과 자신의 실행 로그·파형을 함께 설명한다.

통합 TB · 경계와 선택 순서

tb_lab1_integrated.sv · 61-68행 · 실제 VS Code 화면

```
61      3:expected=(((n/16)<(n%16))?16:0)|(((n/16)-(n%16))&15))<<3;  
62      4:expected=(((n/16)>(n%16))?4:(((n/16)==(n%16))?2:1))<<5;  
63      5:expected=(((n/16)>>(3-(n%4)))&1)<<7;  
64      6:expected=(n>=128)?(128>>(n%8)):0;  
65      7:begin for(k=0;k<8;k=k+1)if(n==(128>>k))expected=k<<5;end  
66      8:expected=1<<(n%8);  
67      9:begin expected=digits[n%16];expected_seg=expected;end  
68      endcase
```

감산 borrow, 비교 결과, 선택순 순서, 인코더의 비정상 입력까지 기대값에 반영한다.

실험 전 레포트에는 이 코드의 역할과 자신의 실행 로그·파형을 함께 설명한다.

통합 TB · 출력과 모드 번호

tb_lab1_integrated.sv · 69-79행 · 실제 VS Code 화면

```
69     if(led!=expected || seg_data!=expected_seg)
70     | $fatal(1,"mode=%0d sw=%h expected LED=%h got=%h",m,sw,expected,led);
71     checked=checked+1;
72     end
73     // Allow two complete 16x2 refreshes, including a mode change midway through a frame.
74     cycles(350);
75     expected_heading="MODE 00";
76     expected_heading[15:8]=(m==9)?8'h31:8'h30;
77     expected_heading[7:0]=(m==9)?8'h30:(8'h31+m);
78     if(completed_line1[127:72]!=expected_heading || init_count!=5)
79     | $fatal(1,"LCD mode text mismatch: %s",completed_line1);
```

LED·7세그먼트를 매 입력마다 비교한다. 화면 갱신을 기다린 뒤 LCD 첫 줄의 MODE 번호와 초기화 횟수를 검사한다.

실험 전 레포트에는 이 코드의 역할과 자신의 실행 로그·파형을 함께 설명한다.

통합 TB · 회로명 대조

tb_lab1_integrated.sv · 80~87행 · 실제 VS Code 화면

```
80  case(m)
81    0:expected_name="AND OR XOR      ";1:expected_name="FULL ADDER      ";
82    2:expected_name="4 BIT ADDER     ";3:expected_name="4 BIT SUBTRACT  ";
83    4:expected_name="4 BIT COMPARE   ";5:expected_name="4 TO 1 MUX     ";
84    6:expected_name="1 TO 8 DEMUX    ";7:expected_name="8 TO 3 ENCODER  ";
85    8:expected_name="3 TO 8 DECODER  ";9:expected_name="HEX 7 SEGMENT ";
86  endcase
87  if(completed_line2!=expected_name)$fatal(1,"LCD circuit text mismatch: %s",completed_line2);
```

두 번째 줄은 16문자 전체를 해당 모드의 기대 회로명과 비교한다.
번호만 맞고 이름이 틀린 오류도 잡는다.

실험 전 레포트에는 이 코드의 역할과 자신의 실행 로그·파형을 함께
설명한다.

통합 TB · 길게 누름 · 순환 · 복귀

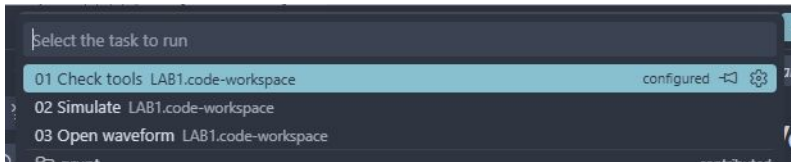
tb_lab1_integrated.sv · 88-99행 · 실제 VS Code 화면

```
88 // A long press advances once and only once.
89 mode_button=1;cycles(80);assert_mode((m==9)?0:m+1);
90 mode_button=0;cycles(12);
91 end
92 assert_mode(0);press_once;assert_mode(1);
93 init_count=0;rst=1;cycles(3);rst=0;cycles(200);assert_mode(0);
94 if(init_count!=5 || completed_line1[127:72]!="MODE 01")$fatal(1,"LCD reset recovery failed");
95 if(checkled!=2560 || frames<10)$fatal(1,"Incomplete integrated test");
96 $display("LAB1_PASS lab1_integrated cases=%0d",checked);$finish;
97 end
98 initial begin #1000000;$fatal(1,"Watchdog");end
99 endmodule
```

80클럭 동안 눌러도 한 번만 넘어가는지, 10→01 순환과 재리셋 후 LCD 복구를 검사한다.

2,560개 출력과 10개 이상 화면을 확인한 뒤 PASS를 출력한다. 외부 보드의 실제 동작은 별도 실험이다.

세 작업을 순서대로 실행



1. Terminal → Run Task... → 01 Check tools.
 2. 02 Simulate → 종료까지 기다린다.
 3. PROJECT/build/vscode/simulation.log를 연다.
 4. LAB1_PASS lab1_integrated cases=2560을 확인한다.
 5. 03 Open waveform으로 wave.vcd를 연다.
- CLI workspace는 Icarus Verilog를 선택한다.

통합 테스트의 검사 범위

10개 모드 × DIP 256개 = 2,560개 출력 비교.

reset → MODE 01, 짧은 바운스 무시, 길게 눌러도 한 번만 전환, 10→01 순환을 검사한다.

LCD 초기 명령과 외부 버스에서 완성된 두 줄의 번호·이름을 비교한다.

TB는 10ns 클록, 디바운스 4클록, 전원 대기 5클록으로 시간을 줄인다.
실제 합성은 1kHz 보드 설정이다.

XSim 검사 종료: 72430ns. 실제 보드의 전압·LCD 연결은 별도로 확인한다.

VaporView에서 통합 파형 읽기

1. wave.vcd가 텍스트면 탭 우클릭 → Reopen Editor With... → VaporView.
2. Netlist에서 tb_lab1_integrated를 펼친다.
3. clk, rst, mode_button, sw, led, seg_data를 추가한다.
4. dut 안의 mode와 lcd_e, lcd_rs, lcd_rw, lcd_data를 추가한다.
5. Zoom to Fit 뒤 모드가 바뀌는 구간을 확대한다.
6. 버튼 입력부터 모드 변경까지의 지연과 LCD 한 화면 갱신을 구분한다.

실험 전 레포트에 넣을 것

1. 모드별 예상 입력·출력과 비트 순서를 표로 작성한다.
2. 10개 회로·버튼·LCD 파일의 역할을 설명한다.
3. PASS 로그, VCD, 모드 전환 및 LCD 파형 캡처를 연결한다.
4. TB에서 줄인 시간과 실제 1kHz 설정을 구분한다.
5. 실험에서 확인할 버튼·LCD·DIP·LED 장면을 계획한다.

레포트 양식과 예시

통합 구조와 입력 배치

lab1_integrated → 10개 조합회로 + button_onepulse + lcd_modes.

clk=B6(1kHz), KEY1=reset, KEY2=mode_button.

DIP1-8은 sw[7:0], LED1-8은 led[7:0]에 대응한다.

개별 실습의 KEY 입력은 통합본에서 DIP로 옮겨 모드 버튼과 충돌하지 않게 한다.

단일 7세그먼트는 모드 10에서만 동작한다.

통합 동작 명세와 핀표

모드별 조작 1-5

모드	입력	출력
01	a=DIP1, b=DIP2	LED1=AND, 2=OR, 3=XOR
02	a,b,cin=DIP1,2,3	LED1=carry, 2=sum
03	a=DIP1-4, b=DIP5-8	LED1=carry, 2-5=sum
04	a=DIP1-4, b=DIP5-8	LED1=borrow, 2-5=diff
05	a=DIP1-4, b=DIP5-8	LED1=콤, 2=같음, 3=작음

사용하지 않는 LED 비트는 0이다. reset하면 모드 01.

모드별 조작 6-10

모드	입력	출력
06	i=DIP1-4, s=DIP7-8	LED1=i[3-s]
07	i=DIP1, s=DIP6-8	s=0→LED1, s=7→LED8
08	DIP1-8 중 하나만 1	LED1-3: DIP1→0, DIP8→7
09	abc=DIP6,7,8	0→LED8, 7→LED1
10	hex=DIP5-8	a-dp 패턴, 1이면 점등

사용하지 않는 LED 비트는 0이다. reset하면 모드 01.

버튼을 길게 누르면

2단 동기화 → 안정된 입력 20클록 확인 → 상승 시 1클록 펄스.

1kHz에서 약 20ms 안정 시간에 동기화 지연이 추가된다.

누른 채 유지하면 한 번만 바뀐다. 땀 뒤 다시 눌러야 다음 모드가 된다.

모드 10 다음은 01. reset은 즉시 01로 돌아간다.

실험 영상에서 짧게 누르기·길게 누르기·순환을 각각 보여준다.

LCD가 표시되는 과정

전원 대기 50ms → 38,38,38,0C,06 초기 명령 → 두 줄 반복 갱신.

첫 줄 MODE 01...MODE 10, 두 번째 줄 회로 이름.

E high는 1ms, 바이트 간격은 최소 4ms. 첫 초기 명령 간격은 더 길다.

한 화면 시작 때 모드를 저장하여 두 줄이 다른 모드로 섞이지 않게 한다.

새 모드는 현재 전송을 마친 뒤 갱신되므로 잠깐 기다리고 읽는다.

CLI 실행 환경

macOS ARM64 바이너리 배포를 확인하고 WSL Ubuntu-26.04에서 Linux x86-64 바이너리를 실행했다.

제작자에게 Mac이 없으므로 Mac 실행·보드 기록을 검증했다고 쓰지 않는다.

API0 1.5.1, OSS CAD Suite 2026.08.19, openXC7 2026.08.20.

Yosys 0.63+173 / nextpnr-xilinx 68aeeb3 / Icarus Verilog 14.0 개발 버전.

openXC7 공식 ARM64·Linux 배포

설치와 경로

Python 3의 venv를 만들고 `pip install apio==1.5.1`을 실행한다.

`apio packages install`로 `oss-cad-suite`와 `openxc7`을 준비한다.

도구 경로는 `./apio/packages/oss-cad-suite/bin` 및 `openxc7/bin`이다.

`LD_LIBRARY_PATH`를 두 묶음의 `lib`로 전역 지정하지 않는다. 각 배포의 실행 래퍼가 자신의 라이브러리를 선택한다.

macOS는 `darwin-arm64`, WSL은 `linux-x86-64` 자산을 사용한다.

복사 가능한 전체 설치·실행 명령

대상 부품 데이터베이스

대상은 xc7s75fgga484-1이다. xc7s50으로 바꾸지 않는다.

배포에 포함된 S50 chipdb만으로는 S75를 빌드할 수 없다.

openXC7/prjxray-db의 spartan7 자료와 bbaexport.py·bbasm으로 정확한 부품의 chipdb를 준비한다.

데이터베이스 버전, 내보내기 결과와 실패 로그를 보관한다.
공식 디바이스 데이터베이스

실제 실행 결과와 남은 제약

Icarus: 2,560개 통과. Yosys: 논리 게이트·통합 합성 성공.

nextpnr: 논리 게이트 K4, 통합 클록 B6 핀을 찾지 못해 실패. 프레임·bit 단계는 실행하지 못했다.

검증한 S75 DB의 필수 핀 38개 중 28개가 빠져 있다. S75 tilegrid는 S50 파일과 바이트 단위로 같았다.

Vivado 공식 부품 데이터에는 K4와 B6가 존재한다. 다른 핀·다른 FPGA로 바꾸어 성공 처리하지 않는다.

CLI bit 목표는 미완료이며, 실행 로그·DB 해시·후속 작업을 공개한다. 실패 근거와 데이터베이스 대조

검증 가능한 순서로 실행

1. 02 Simulate: Icarus Verilog의 2,560개 검사와 VCD.
2. Yosys: synth_xilinx로 RTL을 Xilinx 셀에 매핑.
3. nextpnr-xilinx: 정확한 chipdb와 XDC로 배치·배선.
4. fasm2frames: FASM을 디바이스 프레임으로 변환.
5. xc7frames2bit: 정확한 part.yaml로 bit 생성.

각 단계의 입력·출력·종료 코드를 확인한다. 앞 단계 실패 후 오래된 산출물을 사용하지 않는다.

실행 명령과 결과 확인

프로젝트 폴더: opensource_cli/integrated.

```
python3 ../../tools/lab1.py simulate -project . -simulator iverilog
```

```
python3 ../../tools/openxc7_build.py -project .
```

생성 명령·실행 로그·상태·bit 해시는 프로젝트 build/cli와 검증 문서에서 확인한다.

합성 성공만으로 배치배선·bit 생성 완료를 선언하지 않는다.
실제 단계별 결과

대상 부품·핀 제약 (1)

xc7s75fgga484-1 · IOSTANDARD=LVCMOS33

포트	PACKAGE_PIN
clk	B6
rst	K4
mode_button	N8
sw[0]	U4
sw[1]	V4
sw[2]	W1

XDC에서 포트·핀 번호를 대조한다. 다른 보드에는 그대로 쓰지 않는다.

대상 부품·핀 제약 (2)

xc7s75fgga484-1 · IOSTANDARD=LVCMOS33

포트	PACKAGE_PIN
sw[3]	W4
sw[4]	T1
sw[5]	U2
sw[6]	W3
sw[7]	Y1
led[0]	N5

XDC에서 포트·핀 번호를 대조한다. 다른 보드에는 그대로 쓰지 않는다.

대상 부품·핀 제약 (3)

xc7s75fgga484-1 · IOSTANDARD=LVC MOS33

포트	PACKAGE_PIN
led[1]	M1
led[2]	M3
led[3]	M7
led[4]	N7
led[5]	M2
led[6]	M4

XDC에서 포트·핀 번호를 대조한다. 다른 보드에는 그대로 쓰지 않는다.

대상 부품·핀 제약 (4)

xc7s75fgga484-1 · IOSTANDARD=LVC MOS33

포트	PACKAGE_PIN
led[7]	L4
seg_data[0]	R6
seg_data[1]	R4
seg_data[2]	R2
seg_data[3]	T5
seg_data[4]	N3

XDC에서 포트·핀 번호를 대조한다. 다른 보드에는 그대로 쓰지 않는다.

대상 부품·핀 제약 (5)

xc7s75fgga484-1 · IOSTANDARD=LVC MOS33

포트	PACKAGE_PIN
seg_data[5]	P7
seg_data[6]	P3
seg_data[7]	P1
lcd_data[0]	A4
lcd_data[1]	B2
lcd_data[2]	C3

XDC에서 포트·핀 번호를 대조한다. 다른 보드에는 그대로 쓰지 않는다.

대상 부품·핀 제약 (6)

xc7s75fgga484-1 · IOSTANDARD=LVCMOS33

포트	PACKAGE_PIN
lcd_data[3]	D4
lcd_data[4]	A2
lcd_data[5]	C5
lcd_data[6]	C1
lcd_data[7]	D1
lcd_e	A6

XDC에서 포트·핀 번호를 대조한다. 다른 보드에는 그대로 쓰지 않는다.

대상 부품·핀 제약 (7)

xc7s75fgga484-1 · IOSTANDARD=LVC MOS33

포트	PACKAGE_PIN
lcd_rs	G6
lcd_rw	D6

XDC에서 포트·핀 번호를 대조한다. 다른 보드에는 그대로 쓰지 않는다.

보드 실험으로 이어갈 때

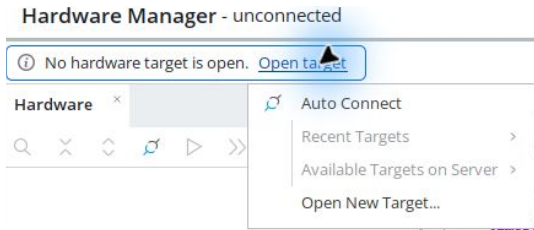
현재 검증한 CLI 흐름은 S75 데이터베이스의 핀 누락으로 bit를 생성하지 못했다.

보드 실험은 같은 RTL의 Vivado 통합 프로젝트에서 생성한 bit로 진행한다.

CLI 실행 결과와 Vivado bit의 생성 경로·해시를 구분하여 레포트에 적는다.

Vivado 통합 매뉴얼

실제 보드 연결



Open Hardware Manager → Open target → Auto Connect.

KEY1=reset, KEY2=다음 모드. DIP1-8=sw[7:0]. LED1-8=led[7:0].

장치가 없으면 보드 전원·USB JTAG·드라이버·VM USB 연결을 점검한다.

Program Device와 동작 확인

1. 연결된 장치 이름이 xc7s75인지 확인한다.
2. 장치 우클릭 → Program Device.
3. 이번 프로젝트의 lab1_integrated.bit를 선택하고 Program.
4. 기록 완료를 확인한 뒤 입력을 바꾸고 출력과 예상값을 비교한다.
5. 기록 성공 화면과 실제 보드 동작은 각각 증빙한다.

제작 환경의 실제 보드 기록·촬영 검증 여부는 검증표를 따른다. 파일 생성만으로 보드 동작 성공을 선언하지 않는다.

사진과 시연 영상 촬영

KEY1=reset, KEY2=다음 모드. DIP1-8=sw[7:0]. LED1-8=led[7:0].

사진에는 보드 연결과 입력·출력 위치가 함께 보이게 한다.

영상에는 입력을 조작하는 과정과 그에 따른 출력 변화를 담는다.

예상표의 정상·경계 조건을 직접 보여주고 회로 번호를 파일명에 적는다.

통합 영상이라면 각 회로의 타임스탬프를 레포트에서 연결한다.

GitHub 결과 정리

1. reports/pre와 reports/post에 실험 전·후 레포트를 둔다.
 2. evidence/vscode와 evidence/vivado에 로그·파형·캡처를 구분한다.
 3. evidence/board/photos와 videos에 직접 촬영한 자료를 둔다.
 4. 큰 영상·bit는 배포 위치를 정하고 README와 레포트에서 연결한다.
 5. 웹에서 사진 표시와 영상 재생 또는 다운로드가 되는지 확인한다.
- 레포트 양식·예시·GitHub 안내

실험 후 레포트

1. Vivado 버전·part·top·핀 제약·코드 커밋을 기록한다.
2. VS Code와 Vivado의 입력·출력·시간·검사 수를 비교한다.
3. 합성·구현·bit 경로와 경고·수정 사항을 설명한다.
4. 장치 기록 화면·사진·영상과 해당 조건을 연결한다.
5. 예상값·두 시뮬레이션·실측의 일치 또는 차이 원인을 해석한다.
6. 미수행 항목은 미완료로 남기고 후속 확인을 적는다.

전체 목차 PDF 프로젝트로 돌아가기